DevSecOps Pipeline — Step-by-Step Documentation

Sep 28, 2026 ·

Overview

The goal: catch vulnerabilities **before** an application reaches production by embedding automated security checks at every CI/CD stage ("shift left"), with hard gates that block deployment when critical issues are found.

Tools used

Purpose	Tool
Static code analysis (SAST)	SonarQube
Dependency vulnerability scanning	OWASP Dependency-Check
Container/image scanning	Trivy
Secret detection	Gitleaks
Secret management	HashiCorp Vault
Image signing/verification	cosign
Pipeline orchestration	Jenkins
Cluster-level signature enforcement	Kyverno (optional, on top of Jenkins)

Each of the 10 tasks below maps to one or more named stages in the project's `Jenkinsfile`, so this document and the runnable code stay in lockstep.

Prerequisites and repo layout

- A Jenkins server (or Azure DevOps, with adaptation) reachable from wherever the app source lives
- Docker installed on the build agent
- A container registry to push images to

- A Kubernetes cluster for the deploy stage (can be added later)

The delivered project (`devsecops-pipeline.zip`) is laid out as:

File	Purpose
`Jenkinsfile`	the pipeline itself, all 10 stages
`sonar-project.properties`	SonarQube scan config
`dependency-check-suppression.xml`	reviewed false-positive list
`.gitleaks.toml`	secret-scanning rules
`vault/policy.hcl` , `vault/setup-approle.sh`	Vault access policy and bootstrap script
`k8s/deployment.yaml`	deploy manifest
`k8s/cosign-admission-policy.yaml`	cluster-level "signed images only" enforcement
`scripts/generate_security_report.py`	merges scan results into one HTML report

This project is the pipeline only — it does not include a sample application. It expects the repo it runs against to already contain application source and a `Dockerfile` .

Step 1 — Configure source code scanning (SonarQube)

1. Start a SonarQube server: `docker run -d --name sonarqube -p 9000:9000 sonarqube:lts-community`
2. Open `http://localhost:9000` (default login `admin` / `admin` , then reset the password)
3. Generate a token: **My Account → Security → Generate Token**
4. In Jenkins: **Manage Jenkins → System → SonarQube servers**, add a server named `sonarqube-server` , store the token as credential id `sonar-token`
5. Install the **SonarQube Scanner** and **Sonar Quality Gate** Jenkins plugins
6. `sonar-project.properties` in the repo defines what gets scanned — adjust `sonar.sources` and coverage report paths for your stack

Pipeline stage **"1. Static Code Analysis"** runs `sonar-scanner` against the repo on every build.

Step 2 — Scan dependencies (OWASP Dependency-Check)

1. Install the Dependency-Check CLI on the build agent, or install the Jenkins plugin
2. `dependency-check-suppression.xml` holds reviewed false positives only — every entry needs a reviewer note and date
3. Pipeline stage **"2. Dependency Scan"** runs the scan and fails the build if any dependency has a CVSS score ≥ `DEPCHECK_CVSS_FAIL_THRESHOLD` (default `7`, i.e. High/Critical)
4. Reports land in `security-reports/dependency-check/` in HTML, XML, JSON and SARIF

Step 3 — Scan Docker images (Trivy)

1. Install Trivy on the Jenkins agent: `curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh -s -- -b /usr/local/bin`
2. Stage **"Build Docker Image"** builds the application image
3. Stage **"3. Container Image Scan"** runs Trivy against that image and fails on any CRITICAL/HIGH finding (`TRIVY_SEVERITY_FAIL`)
4. Also scan the base image on a schedule outside the pipeline (`trivy image <base>`) to catch newly disclosed CVEs in images not yet rebuilt

Step 4 — Detect exposed secrets (Gitleaks)

1. Install Gitleaks on the build agent (`brew install gitleaks` or the binary from its GitHub releases)
2. `.gitleaks.toml` extends the built-in rule set (AWS keys, private keys, tokens, generic API keys) — add org-specific patterns as needed
3. Stage **"4. Secret Scanning"** fails the build immediately on any match
4. Also run Gitleaks as a pre-commit hook and enable GitHub secret scanning / push protection, so secrets are caught before they reach CI at all

Step 5 — Configure security quality gates

Two gates are enforced:

- **SonarQube Quality Gate** — code smells, bugs, vulnerabilities, coverage, duplication; configured in SonarQube under **Quality Gates**, attached to the project. Stage **"5a. SonarQube Quality Gate"** waits on it and blocks the pipeline if it fails.
- **Scanner-level thresholds** — `TRIVY_SEVERITY_FAIL` and `DEPCHECK_CVSS_FAIL_THRESHOLD` environment variables at the top of the `Jenkinsfile`.

Tune both to your organization's risk tolerance — start strict (fail on Critical/High) and relax deliberately rather than the reverse.

Step 6 — Block deployment when critical checks fail

Every scanning stage either exits non-zero (Trivy `--exit-code 1`, Dependency-Check `--failOnCVSS`, Gitleaks `--exit-code 1`) or calls Jenkins' `error()` step. Jenkins' declarative pipeline stops at the first failed stage by default, so a critical finding anywhere upstream means the deploy stage never runs — no manual judgment call is needed to stop a bad release.

Step 7 — Store credentials in a secret manager (Vault)

1. Run `vault/setup-approle.sh` once against your Vault server — it creates a scoped policy (`vault/policy.hcl`), an AppRole for Jenkins, and seeds an example secret at `secret/my-app/registry`
2. Store the generated Role ID / Secret ID as a Jenkins username/password credential named `vault-approle` — this is the only long-lived secret Jenkins holds
3. Stage **"7. Fetch Credentials from Vault"** logs in via AppRole, reads the registry credentials, and discards them at the end of the stage — nothing is written to disk or logs

Azure Key Vault alternative: use the `azure-keyvault` Jenkins plugin or `az keyvault secret show`, authenticating via a managed identity — same "fetch at runtime, never store" pattern.

Step 8 — Configure image signing and verification (cosign)

1. Generate a keypair once: `cosign generate-key-pair` → produces `cosign.key` (private) and `cosign.pub` (public)

2. Store `cosign.key` as a Jenkins Secret file credential (`cosign-private-key`) and its passphrase as a Secret text credential (`cosign-password`); commit `cosign.pub` to the repo — it's safe to be public
3. Stage **"8. Sign Image"** signs the pushed image; stage **"8a. Verify Image Signature"** immediately re-verifies it, catching signing misconfiguration before it reaches deploy
4. For defense in depth, `k8s/cosign-admission-policy.yaml` installs a Kyverno `ClusterPolicy` so the cluster itself refuses to run any pod whose image isn't signed by your key — even if Jenkins is bypassed entirely

Step 9 — Generate security reports

1. `scripts/generate_security_report.py` merges the SonarQube, Dependency-Check, Trivy, Gitleaks and cosign results into one `security-summary.html`
2. Stage **"9. Generate Security Report"** runs it, then `archiveArtifacts` attaches all raw plus summary reports to the Jenkins build, and `publishHTML` (requires the HTML Publisher plugin) makes the summary viewable from the build page
3. Point stakeholders (security team, auditors) at the Jenkins build's "DevSecOps Security Report" link rather than raw scanner output

Step 10 — Deploy only approved artifacts

1. The **Approval Gate** stage pauses the pipeline (only on `main`) and requires a human in the `devsecops-approvers` Jenkins group to click Approve — this only happens once every automated gate above has already passed
2. Stage **"10. Deploy Approved Artifact"** re-verifies the cosign signature immediately before `kubectl apply`, then applies `k8s/deployment.yaml` with the built image substituted in
3. Combined with the Kyverno admission policy from Step 8, there are two independent enforcement points: Jenkins (process) and the cluster (runtime) — an unsigned or unscanned image cannot reach production even if one control is bypassed

How to run the pipeline

Local setup (Docker)

1. **Unzip and push the project to a git repo:**

```
unzip devsecops-pipeline.zip && cd devsecops-pipeline
git init && git add . && git commit -m "Initial DevSecOps pipeline"
git remote add origin <your-repo-url> && git push -u origin main
```

2. **Start SonarQube:** `docker run -d --name sonarqube -p 9000:9000 sonarqube:lts-community`
3. **Start Vault (dev mode):**

```
docker run -d --name vault -p 8200:8200 -e 'VAULT_DEV_ROOT_TOKEN_ID=root'
hashicorp/vault:latest
export VAULT_ADDR='http://127.0.0.1:8200' VAULT_TOKEN='root'
bash vault/setup-approle.sh
```

4. **Start Jenkins with Docker access:**

```
docker volume create jenkins_home
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home -v
/var/run/docker.sock:/var/run/docker.sock \
  -u root jenkins/jenkins:lts
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

5. **Install the security tools inside the Jenkins container** (Trivy, Dependency-Check, Gitleaks, cosign, Vault CLI, kubectl) and the Jenkins plugins: SonarQube Scanner, Sonar Quality Gate, Dependency-Check, HTML Publisher, Credentials Binding
6. **Generate a cosign keypair** and commit `cosign.pub` to the repo
7. **Add Jenkins credentials** (`sonar-token` , `vault-approle` , `cosign-private-key` , `cosign-password` , `kubeconfig-prod`) and register the `sonarqube-server` connection
8. **Create the pipeline job:** New Item → Pipeline → Pipeline script from SCM → point at the repo, script path `Jenkinsfile`

Triggering a run

- **UI:** open the job → **Build Now** → watch **Console Output**
- **CLI:** `curl -X POST http://localhost:8080/job/devsecops-pipeline/build --user admin:<api-token>`
- **On every push:** enable a GitHub webhook to `http://<jenkins-host>:8080/github-webhook/` , or **Poll SCM**

Watching it run

The **Stage View** plugin shows all 10 stages as colored boxes — green means passed, red marks exactly where a security gate blocked the build. On `main`, once every scan passes, the pipeline pauses at **Approval Gate** until someone in `devsecops-approvers` clicks **Proceed**.

Note: for a first local dry run, either comment out the **Push Image**, signing, and **Deploy** stages, or point `REGISTRY` at a local registry (`docker run -d -p 5000:5000 registry:2`) and `kubeconfig-prod` at a local cluster (`kind` or `minikube`).

Quick-start checklist

- [ ] SonarQube server reachable from Jenkins, `sonar-token` credential created
- [ ] Dependency-Check CLI/plugin installed on the agent
- [ ] Trivy installed on the agent
- [ ] Gitleaks installed on the agent
- [ ] Vault server reachable, AppRole created (`vault/setup-approle.sh`), `vault-approle` credential added
- [ ] cosign keypair generated, `cosign-private-key` + `cosign-password` credentials added, `cosign.pub` committed
- [ ] `kubeconfig-prod` credential added for the target cluster
- [ ] HTML Publisher + Dependency-Check + SonarQube Scanner Jenkins plugins installed
- [ ] `devsecops-approvers` Jenkins group created for the approval gate
- [ ] (Optional) Kyverno installed and `k8s/cosign-admission-policy.yaml` applied